COMP 3415 - Software Engineering

P2: Engineering Baseline - MemoryCircle

Team FrameFour
Oluwatolayomi Akande, Daniel Seriki, Nimo Usman, Franklin Uba
October 27, 2026

GitHub: https://github.com/osakande-lab/memorycircle
Final implementation branch: main

1. Implementation summary

MemoryCircle helps users keep photos and the stories behind them. For P2, we used our existing Expo/React Native app as the starting point and completed a local memory workflow. A user can create a memory, attach photos, reopen saved memories, select a collage layout and play a photo reel with chosen audio.

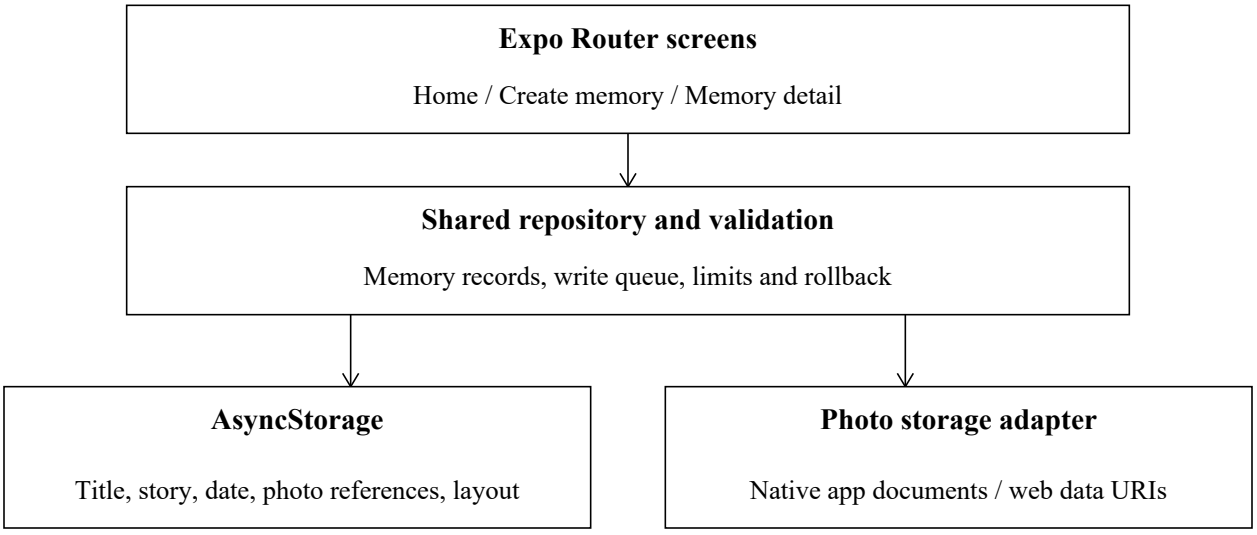
This is a change from our P1 implementation plan, which proposed React, Express and Supabase for private group sharing. The current increment uses TypeScript, Expo Router, AsyncStorage and device storage. It gives us working photo-input and presentation functionality, but accounts, real groups and generated video files are still future work. The circle card is labelled as a future feature.

Feature	Working implementation
Create and browse memories	Required title/story, recent-memory list, detail screen, saved date and visible empty/error states.
Attach photos	Photo picker and previews; ordered local persistence; validation and cleanup when saving fails.
Collage selection	Classic, Grid and Scrapbook layouts; the selected layout is saved with the memory.
Photo reel and audio	Images advance every 2.5 seconds; local audio can be selected, played and stopped. Navigating away pauses playback.

The title limit is 80 characters and the story limit is 2000. A memory can contain up to 10 JPEG/PNG photos, each within 10 MB and 20 megapixels. The local prototype limits a device to 30 photos per UTC month. These are device-level checks, not P1's future per-member/group enforcement.

The reel is an in-app slideshow. It does not encode, automatically schedule or download an MP4. We have kept that distinction in the interface, backlog and demonstration.

2. Initial architectural design



Home loads saved memories when the screen receives focus. Create handles the form and photo picker. The detail route looks up one memory by its ID and manages collage selection, reel timing and audio playback. Expo Router handles navigation between these screens.

Module	Responsibility
src/data/memory-repository.ts	Validates input, reads stored records, serializes writes, creates memories, saves layouts and handles failures.
src/data/repository.ts	Connects the shared repository to AsyncStorage and the platform photo adapter.
photo-store.native.ts / photo-store.web.ts	Copies native photos to app documents, or converts browser blobs to persistent data URIs.
src/app/memory/[id].tsx	Displays the memory, persists collage choices and controls the timer, document picker and platform audio component.

We separated persistence from the screens so the same save rules can be tested without a phone. Writes share a queue within one app runtime, preventing overlapping saves from losing each other's changes. Photos are saved before metadata is published. On failure, copied files are removed where possible and no completed memory is added.

Native file URIs and browser blob URLs behave differently, so they use separate adapters. Web images must survive a reload; storing only a temporary blob URL would break them. The web build uses client-side rendering because audio and device-local data are available at runtime, not during static server rendering.

This design has no backend authorization or multi-device transaction. Local storage is not a substitute for the private-group security requirements in P1.

3. Testing plan and results

We used Node's test runner for the shared repository. Tests inject storage and photo-copy adapters to check normal behaviour and controlled failures. They call the same validation and repository functions used by the screens. We also ran TypeScript checking, linting, a production web export and a browser walkthrough.

Test area	Cases covered
Memory persistence	Trimmed valid input, reopen, empty state, newest-first order, overlapping saves, corrupt records and older records without photo/layout fields.
Validation and limits	Blank/overlong text, photo count, format, file size, pixel dimensions, monthly quota and UTC month rollover.
Failure handling	Photo-copy failure, metadata/storage-quota failure, cleanup failure and recovery after a rejected queued write.
Layout and reel	All three layout values persist without changing content; invalid/failed layout writes preserve data; reel index advances and wraps.

Automated result: 24 tests passed, 0 failed. TypeScript checking passed. The production web export passed after changing the app from static prerendering to client-side rendering. Linting passed with no errors or warnings. Commands and output are available in the README and docs/test-results.tap.

Example of requirement, implementation and verification

P2-M01: As a user, I want to create and reopen a memory so I can keep its title and story. Acceptance criteria: a valid nonblank title/story is trimmed and saved with an ID/date; it appears in the recent list and reopens with the same data. Blank/overlong input creates no record.

Implementation: validateDraft and create/list/find in memory-repository.ts are called by the create, home and detail screens. **Verification:** the persistence test recreates a repository using the stored value and finds the same memory. Separate blank-title and blank-story tests confirm that no value is written. The browser walkthrough confirms the visible create-and-reopen path.

Manual checks and boundaries

In the browser, we submitted an empty form, created a memory with three synthetic photos, reopened it, changed to Grid and refreshed the detail route. The title, story, images and layout remained; all three images loaded. We also exercised reel controls and selected the supplied audio sample.

Repository tests do not verify every native picker, file-permission or audio behaviour. Native-device checks and the complete Chrome/Firefox/keyboard matrix remain further validation. There is no cloud authorization, MP4 generation, hosted load or worker-recovery result to claim for this increment.

4. Updated product backlog

The original P1 backlog remains visible below. Its estimates total 47 points. We have not marked a large P1 story Done just because a local part exists. Our working P2 tasks refine the current increment: P2-M01 create/reopen; P2-M02 attach/persist photos; P2-M03 collage preference; P2-M04 reel/audio. They are tracked separately from the original total and are complete for the reviewed local/web baseline; native-device rehearsal remains.

P1 ID / story	Priority	Points	Current status
US-01 Account access	Blocker	3	To Do
US-02 Private group	Blocker	3	To Do
US-03 Invite/join	Critical	5	To Do
US-06 Photo upload	Critical	5	In Progress - local photo input
US-08 Monthly timeline	Critical	3	In Progress - local memory list
US-04 Leave/remove	Critical	5	To Do
US-07 Manage photos	Critical	3	To Do
US-10 Automatic recap	Critical	8	To Do
US-11 Manual recap/status	Critical	5	To Do
US-12 Private MP4 delivery	Critical	3	To Do
US-09 Select music	Major	2	In Progress - session audio
US-05 Posting schedule	Major	2	To Do

Changes since P1

We changed the initial implementation from cloud group sharing to a local Expo memory workflow, using the app already built by the team. Local saving, photo presentation and playback became the focus of this increment. The four P2 tasks make that scope explicit; they do not change the meaning of P1's authentication, membership and video requirements.

US-06 has local photo selection, storage and limits, but no authenticated group upload or server validation. US-08 has a recent-memory list, but no shared monthly timeline. US-09 has a local audio picker/player, but no licensed catalogue or persisted music preference. The original estimates are retained until we review the backend work again.

The next priorities are account access and private-group membership, followed by shared storage and a real recap pipeline. A local slideshow does not complete the P1 rendering spike or the automatic-video story.

5. Iteration review

Planned: P1 selected accounts, private groups and photo uploads (11 points), plus an FFmpeg/hosting spike. **Completed:** the revised Expo increment supports local create/save/reopen, validated photo persistence, collage/reel/audio, tests and setup documentation. Cloud and hosting work remains incomplete.

Team contributions

All four members reviewed and accepted this increment. We worked together, with Oluwatolayomi operating GitHub through osakande-lab. The commits therefore appear under one account. Responsibilities were divided as follows:

Member	Responsibilities for this increment
Oluwatolayomi Akande	Integration, requirement/backlog updates, report and demonstration coordination.
Daniel Seriki	Memory persistence, navigation/detail integration and review of storage behaviour.
Nimo Usman	Photo selection, platform storage, input limits and failure-handling review.
Franklin Uba	Collage, reel/audio controls, automated checks and demonstration preparation.

Difficulties and decisions

We moved memory-array writes from individual screens into a shared repository to handle overlapping saves and consistent validation. Copy and metadata failures now trigger cleanup rather than publishing an incomplete memory.

Native files and browser blobs needed separate storage adapters. Static web export also failed when it tried to use the browser Audio API during rendering; client-side output resolved this. We separated nested playback controls and fixed SDK type errors.

Next iteration

We will implement accounts and real private groups, define the migration from local records to shared storage, and move privacy and quotas to server-side checks. We will then add persistent music choices and actual video generation, with job status, retry and protected delivery. Native-device tests and browser/accessibility checks will continue as these features develop.

Repository and TA review

The repository contains the app, shared storage module, tests, setup instructions and updated backlog. The TA demonstration follows create, validate, save, reopen, change layout, play/stop reel/audio, then show architecture, tests and backlog. The separate TA review guide gives each member speaking notes, exact actions and likely questions. GitHub should be shown on main, including merged PR #2, so the inspected code matches this report.